C++ | Classes | Constructors | Copy Semanti

TABLE OF CONTENTS

LAB A1 Introduction to Classes and Objects

Core OOP Concepts · Classes · Data Members · Member Functions · Encapsulation

1.1 Introduction to OOP

Traditional programming (also called **procedural programming**) designs a program as a sequence of instructions. This works well for small programs, but as programs grow, it becomes very hard to manage and extend.

Object-Oriented Programming (OOP) solves this by modelling a program around **objects** — just like the real world. Each object bundles together its data and the actions it can perform.

Key advantages of OOP:

- Encapsulation – Data and functions are kept together; internal details are hidden.
- Modularity – Programs are split into self-contained, reusable classes.
- Maintainability – Easy to modify or extend without breaking other parts.
- Reusability – A class written once can be used in many programs.
- Scalability – Programs can grow large without becoming unmanageable.

■ EXAM-FOCUSED POINTS

- **What is the difference between procedural programming and OOP?**
- **What is encapsulation and why is it important in OOP?**
- **List three advantages of OOP over traditional programming.**

1.2 Object

An **object** is the fundamental building block of OOP. An object combines:

- **State** – the attributes/data that describe the object (stored in variables called *data members*).
- **Behaviour** – the actions the object can perform (implemented as *member functions*).

Real-world analogy – A Cat:

Attribute (State)	Behaviour
Colour, Breed, Gender, Age, Weight	Sound, Eating, Sleeping, Walking

Formal Definition: In programming, an object is a collection of variables and functions that together have a unique purpose of identifying a distinctive entity.

■ **KEY INSIGHT**

- A class is the blueprint (template); an object is a concrete instance created from that blueprint.
- Multiple objects can be created from a single class, each with its own independent data.

1.3 Data Members

The attributes of an object are stored as **data members** (also called *class variables*). They are declared inside the class body and can be of any standard C++ type (int, float, string, etc.).

Key rules:

- Data members are **private by default** inside a class — they cannot be accessed directly from outside.
- Always refer to them as **data members** or **class variables**, NOT simply as 'variables' — this is an important distinction in OOP terminology.
- A class can have multiple data members of different types.

■■ **COMMON MISTAKES**

- Never call data members just 'variables' in exams — the correct term is 'data member' or 'class variable'.
- Do not try to access private data members directly from outside the class — this causes a compile error.

1.4 Member Functions

The behaviours of an object are implemented as **member functions**. They are declared inside the class and can read or modify the object's data members.

Key rules:

- Member functions can be **overloaded** (same name, different parameters).
- A member function can be defined **inside** the class body (inline) or **outside** using the scope resolution operator.
- The function prototype must always appear inside the class body.

1.4.1 Scope Resolution Operator (::)

When defining a member function **outside** the class, you must tell the compiler which class it belongs to. This is done with the **scope resolution operator** ::.

Syntax:

```
ReturnType  ClassName :: FunctionName( parameters )
{
    // function body
}
```

Complete Example – Cylinder Class:

```
#include <iostream>
using namespace std;

class cylinder
{
    float radius;          // data member (private by default)
    float height;          // data member (private by default)
public:
    void SetRadius(float r);   // function prototype
    void SetHeight();          // function prototype
    void display() const;      // const member function prototype
};

// Separate implementation using scope resolution operator
void cylinder::SetRadius(float r)
{
    radius = r;
}

void cylinder::SetHeight()
{
    cout << "Enter the height: ";
    cin  >> height;
}

void cylinder::display() const    // const function - read only
{
    cout << "Radius : " << radius << endl;
    cout << "Height : " << height << endl;
}

int main()
{
    cylinder obj;
    obj.SetRadius(3.5);
    obj.SetHeight();
    obj.display();
    return 0;
}
```

```
Output: Enter the height: 14
Radius : 3.5
Height : 14
```

Line-by-line explanation:

- float radius; float height; — private data members (no access specifier written, so private by default).
- void SetRadius(float r); — only the prototype is inside the class; body is defined outside.
- void cylinder::SetRadius(float r) — the :: links this definition back to the cylinder class.

- obj.SetRadius(3.5); — calling a public member function using the dot operator on an object.

■ EXAM-FOCUSED POINTS

- **What is the purpose of the scope resolution operator in C++?**
- **Where must the function prototype always appear when using separate implementation?**
- **Can member functions be overloaded? Justify your answer.**

1.5 Constant Member Functions

A **constant member function** (declared with the const keyword) is a **read-only** function. It can read the object's data members but **cannot modify** them.

Syntax:

```
ReturnType FunctionName( parameters ) const
{
    // can only READ data members, not modify them
}
```

Example:

```
void display() const
{
    cout << "The sum is = " << sum;   // reading is allowed
    // sum = 10;   <-- ERROR: modification not allowed in const function
}
```

Why use const member functions?

- They enforce safety — accidentally modifying data is caught at compile time.
- They signal to other programmers that the function is purely for display/retrieval.
- Good OOP design practice: always make display/getter functions const.

■■ COMMON MISTAKES

- Writing const after parameters (not before return type) is the correct syntax.
- Trying to modify any data member inside a const function is a compile-time error.

1.6 Class Definition and Encapsulation

What is a Class?

A **class** is a user-defined data type that serves as a blueprint for creating objects. It groups together data members and member functions into a single unit.

Syntax:

```
class ClassName
{
    // data members and member functions
};   // <-- semicolon is MANDATORY
```

■■ COMMON MISTAKES

- The semicolon after the closing brace of a class definition is MANDATORY. Missing it is a common syntax error.
- A class must be defined BEFORE it can be used to create objects.

What is Encapsulation?

Encapsulation is the OOP principle of **bundling data and functions together** in a single unit (the class) while keeping the internal data **hidden** from the outside world. Access to data is only provided through controlled member functions.

Real-world analogy: Think of a class like a secure room. The data inside is protected. Member functions are the doors — only authorised people (functions) can enter and work with the data.

■ EXAM-FOCUSED POINTS

- **Define encapsulation and explain how it is achieved in C++.**
- **What is the difference between a class and an object?**
- **What is the purpose of the semicolon after a class definition?**
- **Can a class be defined after the main function? What must you do in that case?**

LAB A2 — Access Specifiers, Constructors and Destructors

Public · Private · Constructors · Destructors · Data Hiding

2.1 Access Specifiers

Access specifiers (also called **access modifiers**) control **who can access** the data members and member functions of a class. They are the primary mechanism for enforcing **encapsulation** and **data hiding**.

C++ provides **three** access specifiers:

Specifier	Accessibility	Typical Use
`public`	Accessible anywhere — inside AND outside the class.	Member functions (interface)
`private`	Accessible only inside the class. NOT outside.	Data members (hidden data)
`protected`	Like private, but also accessible by derived classes.	Inheritance (future topic)

2.2 Default Visibility Rules (Critical!)

IMPORTANT RULE: By default, all members of a **class** are **PRIVATE**.
By default, all members of a **struct** are **PUBLIC**.
This is one of the only differences between a struct and a class in C++.

Correct convention: Data members → private; Member functions → public. This provides a safe, controlled public interface.

Example demonstrating access specifiers:

```
#include <iostream>
using namespace std;

class myclass
{
private:
    int datamember;         // cannot be accessed from outside
public:
    int memberfun(int x)    // can be accessed from outside
    {
        datamember = x;     // inside class: private member is accessible
        return datamember;
    }
};

int main()
{
    myclass obj;
    // myclass::datamember = 10; // ERROR: private member!
    obj.memberfun(10);           // OK: public function
    return 0;
}
```

2.3 Constructors

A **constructor** is a special member function that is **automatically called whenever an object is created**. Its primary purpose is to initialise the object.

Rules for Constructors:

- The constructor's name must be **exactly the same as the class name**.
- A constructor **cannot return a value** — not even void (writing void is a syntax error).
- Constructors are generally declared as public.
- Constructors **can be overloaded** (multiple constructors with different parameters).
- The compiler automatically finds and calls the constructor when an object is created.
- Every constructor has a body from which regular member functions can be called.

2.3.1 How the Compiler Identifies a Constructor

When an object is created, the compiler searches for a function inside the class that has **the same name as the class**. That function is the constructor and is called automatically.

Example – Pizza Constructor:

```
#include <iostream>
#include <string>
using namespace std;

class pizza
{
private:
    int size, price, thickness;
    string topping;
public:
    void setsize()    { cout<<"Size: "; cin>>size; }
    void setprice()   { cout<<"Price: "; cin>>price; }
    void setthick()   { cout<<"Thickness: "; cin>>thickness; }
    void settopping() { cout<<"Topping: "; cin>>topping; }

    pizza()   // Constructor: same name as class, no return type
    {
        setsize();
        setprice();
        setthick();
        settopping();
    }

    void display() const
    {
        cout << "Size: "      << size      << endl;
        cout << "Price: "     << price     << endl;
        cout << "Topping: "   << topping   << endl;
        cout << "Thickness: " << thickness << endl;
    }
};

int main()
{
    pizza obj;    // Constructor is called automatically here
    obj.display();
    return 0;
}
```

2.4 Destructors

A **destructor** is a special member function called **automatically when an object goes out of scope** (e.g., when the function ends or the program exits). Its purpose is to perform **cleanup operations** such as freeing dynamically allocated memory.

Rules for Destructors:

- The destructor name = **tilde (~) followed by the class name** → e.g., ~myclass()
- A destructor **takes no arguments** — it cannot be overloaded.
- A destructor **cannot return anything** — not even void.
- There can be only **one destructor** per class.

- Primarily used to free dynamic memory (delete pointers).

Example:

```
class myclass
{
private:
    int datamember;
public:
    myclass()           // Constructor
    {
        cout << "Constructor called!" << endl;
    }

    ~myclass()          // Destructor
    {
        cout << "Destructor called - cleanup done!" << endl;
    }
};

int main()
{
    myclass obj;        // Constructor fires here
    // ... program runs ...
    return 0;           // Destructor fires here when obj goes out of scope
}
```

```
Output:
Constructor called!
Destructor called - cleanup done!
```

2.5 Constructor vs Destructor – Comparison Table

Constructor	Destructor
Called when object is **created**.	Called when object goes **out of scope**.
Can take parameters (overloadable).	Takes **no parameters**. Cannot be overloaded.
Name = class name.	Name = ~ + class name.
Used for initialisation.	Used for cleanup (free memory, close files).
Multiple constructors allowed (overloading).	Only **one** destructor per class.
No return type (not even void).	No return type (not even void).

■ **EXAM-FOCUSED POINTS**

- **What is a constructor? List four rules.**
- **Why can a constructor not have a return type?**
- **What is a destructor and when is it called?**
- **Can destructors be overloaded? Justify.**
- **Write a class with a constructor that prints 'Object Created' and a destructor that prints 'Object Destroyed'.**

LAB A3 Constructor Overloading and Copy Constructors

Nullary · Parameterized · Function Overloading · Default Copy Constructor

3.1 Function Overloading

Function overloading allows two or more functions to share the **same name** as long as they differ in their **parameter list** (number, type, or order of parameters).

How does the compiler choose which function to call?

The compiler examines the arguments passed at the call site and matches them against available prototypes. This is called **function resolution**.

```
int fun(int, float, float);    // version 1: 3 parameters
int fun(int, float);           // version 2: 2 parameters
int fun(float, float, int);    // version 3: different order
```

■■ COMMON MISTAKES

- Two functions with DIFFERENT return types but SAME parameters are NOT considered overloaded — this causes a compile error.
- Only differences in parameter number, type, or order qualify as overloading.

3.2 Constructor Overloading

Constructors can be overloaded just like regular functions. A class can have multiple constructors to allow object creation in different ways.

Types of Constructors:

Nullary Constructor (Parameter-less)	Parameterized Constructor
Takes **no arguments**.	Requires **one or more arguments**.
Called when object created with no arguments: Class obj;	Called when arguments provided: Class obj(val);
Often prompts user for input inside body.	Receives values directly from caller.

Example – Student class with all three constructor types:

```
#include <iostream>
#include <string>
using namespace std;

class student
{
private:
    string name;
    int age;
public:
    student()                   // Nullary constructor
    {
        cout << "Enter name: "; cin >> name;
        cout << "Enter age: ";  cin >> age;
    }

    student(string n, int a)    // Parameterized constructor
    {
        name = n;
        age  = a;
    }

    void showall()
    {
        cout << "Name: " << name << "  Age: " << age << endl;
    }
};

int main()
{
    student s1;                  // Calls nullary constructor
    student s2("Ali", 30);       // Calls parameterized constructor
    student s3(s1);              // Calls DEFAULT COPY constructor

    s1.showall();
    s2.showall();
    s3.showall();                // s3 has same data as s1
    return 0;
}
```

3.3 Default Copy Constructor

A **copy constructor** creates a new object as a **copy of an already existing object**. C++ provides a built-in **default copy constructor** that performs a member-by-member copy.

Two syntaxes to invoke the copy constructor:

```
student s3(s1);    // Function-call notation  — copy constructor called
student s3 = s1;   // Assignment notation     — copy constructor called
```

CAUTION: The copy constructor is called ONLY when a **new object is being created** as a copy. If you assign one existing object to another (after both are already created), the copy constructor is **NOT** called — instead, the assignment operator is used.

Behaviour of the default copy constructor:

- Copies each data member one-by-one from the source object to the new object.
- The original object remains unchanged.
- Works correctly for objects with simple (non-pointer) data members.
- Should NOT be used when objects contain pointer data members (use deep copy instead).

■ EXAM-FOCUSED POINTS

- **What is a copy constructor? When is it called?**
- **What is the difference between a nullary and parameterized constructor?**
- **Write the two syntaxes for calling a copy constructor.**
- **Can a parameterized constructor replace a nullary constructor? Explain.**
- **What happens when you have multiple constructors in one class?**

LAB A4 — Shallow Copy / Deep Copy and Arrays in Classes

Customized Copy Constructors · Pointer Data Members · Static & Dynamic Arrays

4.1 Why Customized Copy Constructors?

Although C++ provides a default copy constructor, you sometimes need to write your **own custom copy constructor**. Reasons include:

- You want to copy only **some** data members (partial copy).
- Your object contains **pointer data members**.
- Your object uses **dynamic memory allocation**.

Custom copy constructor prototype:

```
MyClass (MyClass& other);            // regular copy constructor
MyClass (const MyClass& other);      // const copy constructor (recommended)
```

The parameter **other** is the source object being copied. Using const ensures the source is not accidentally modified.

4.2 Shallow Copy Constructor

A **shallow copy constructor** copies each data member one-by-one from the source to the destination. This is exactly what the default copy constructor does.

When is a shallow copy sufficient?

- The class has NO pointer data members.
- You want to copy only certain static (fixed-size) data members.

Example:

```
class example
{
private:
    int a;
    int b;
public:
    example()               // Simple constructor
    { a = 10; b = 20; }

    example(example& param)  // Shallow copy constructor
    {
        a = param.a;         // copy member a
        b = param.b;         // copy member b
    }

    void showall()
    { cout << "a=" << a << " b=" << b << endl; }
};

int main()
{
    example obj1;            // calls simple constructor: a=10, b=20
    example obj2(obj1);      // calls shallow copy constructor
    obj1.showall();          // a=10  b=20
    obj2.showall();          // a=10  b=20
    return 0;
}
```

4.3 Deep Copy Constructor

A **deep copy constructor** is used when the class has **pointer data members** or uses **dynamic memory**. It allocates **new, independent memory** for the copy.

The Problem with Shallow Copy + Pointers:

When a shallow copy copies a pointer, both the original and the copy point to the **same memory location**. This means:

- Modifying data through one object affects the other object unexpectedly.
- Deleting one object's memory leaves the other with a **dangling pointer** — a dangerous bug.

```
Shallow Copy Problem (Diagram):
Object1 ■■pointer■■■ [Memory Block]
Object2 ■■pointer■■■ [Same Memory Block] ← WRONG!
```

```
Deep Copy Solution (Diagram):
Object1 ■■pointer■■■ [Memory Block A]
Object2 ■■pointer■■■ [Memory Block B] ← Own independent copy
```

Deep Copy Constructor Example (char array):

```
#include <cstring>
class example
{
private:
    char* str;
    int   len;
public:
    example(char* s)                    // Normal constructor
    {
        len = strlen(s);
        str = new char[len + 1];        // allocate memory
        strcpy(str, s);                 // copy content
    }

    example(const example& st)          // DEEP copy constructor
    {
        len = st.len;
        str = new char[len + 1];        // allocate NEW separate memory
        strcpy(str, st.str);            // copy the content, not the pointer
    }

    ~example() { delete[] str; }        // Destructor frees memory
};
```

How to tell if it's deep or shallow copy?

The function **prototype** looks the same for both. The difference is in the **body**:

- Shallow: a = param.a; — direct member copy.
- Deep: ptr = new Type[size]; memcpy(ptr, param.ptr, size); — new allocation + content copy.

Shallow Copy	Deep Copy
Default copy constructor behaviour.	Custom copy constructor required.
Copies the pointer ADDRESS.	Allocates new memory; copies CONTENT.
Both objects share same memory.	Each object has its own independent memory.
Safe for non-pointer data members.	Required for pointer/dynamic data members.
Faster (less overhead).	Safer and correct for dynamic data.

4.4 Working with Arrays in Classes

Arrays are frequently used as data members. There are two types:

4.4.1 Static Array as Data Member

A **static (fixed-size) array** is declared with a known size at compile time.

```
class example
{
private:
    float a[10];       // static array: size is fixed at 10
public:
    example()          // constructor initialises all elements to 0
    {
        for(int i = 0; i <= 9; i++)
            a[i] = 0;
    }
};
```

4.4.2 Dynamic Array as Data Member

A **dynamic array** is allocated at runtime using new. Its size can be determined when the object is created.

```
class example
{
private:
    float* a;          // pointer to dynamic array
public:
    example(int size)  // size passed from main
    {
        a = new float[size];   // allocate memory at runtime
        for(int i = 0; i < size; i++)
            a[i] = 0;
    }

    ~example()         // MUST delete dynamic memory in destructor
    {
        delete[] a;
    }
};
```

4.4.3 Array of Objects

You can create an array where each element is a class object. This is useful for managing multiple instances.

```
int main()
{
    const int size = 10;
    example obj1[size] = example(size);  // array of 10 objects,
                                         // each initialised with constructor
    return 0;
}
```

■ **EXAM-FOCUSED POINTS**

- **Explain the difference between shallow copy and deep copy with a diagram.**
- **Why is shallow copy dangerous when pointer data members are involved?**
- **Write the prototype of a deep copy constructor.**
- **What must you do in the destructor when using dynamic arrays?**
- **What is the difference between a static array and a dynamic array as a data member?**

LAB A5 Friend Functions and Friend Classes

Special Access · Bridging Classes · Forward Declaration

5.1 Why Friend Functions?

One of the goals of OOP is to restrict access to class data. However, there are legitimate scenarios where a function that is **not a member** of a class needs to access its private data. Friend functions provide this capability in a **controlled** way.

Use cases for friend functions:

- A single function needs to access private members of **multiple classes** (acting as a bridge).
- A regular (non-member) function needs access to private class data.
- Operator overloading (advanced topic — relies heavily on friend functions).

5.2 Friend Function – Syntax and Rules

Key Rules:

- The keyword friend is written in the function **prototype inside the class**.
- The keyword friend must **NOT** be written in the function **definition** (outside the class) — doing so is a syntax error.
- Access specifiers (public/private) do **not apply** to friend functions. Placing the prototype in public or private makes no difference.
- Friend functions are **not members** of the class — they are regular functions with special privileges.
- Friend functions are called like regular functions — **NOT** using the dot operator.
- Inside a friend function, both public and private members can be accessed directly.
- Friendship is **one-sided**: the function can access the class, but the class cannot access declarations inside the function.

Syntax:

```
class MyClass
{
private:
    int secretData;
public:
    friend void myFriendFunction(MyClass obj);  // prototype with "friend"
};

// Definition: "friend" keyword NOT written here
void myFriendFunction(MyClass obj)
{
    cout << obj.secretData;   // can access private member directly!
}

int main()
{
    MyClass obj;
    myFriendFunction(obj);    // called like a regular function, NOT obj.fun()
    return 0;
}
```

5.3 Friend Function as a Bridge Between Two Classes

A powerful use of friend functions is to access private members of **two different classes** from a single function. This requires a **forward declaration**.

Forward Declaration: Since C++ reads code top-to-bottom, if class first references class second before second is defined, you must forward-declare second.

Example:

```
class second;                           // Forward declaration

class first
{
private:
    int member1;
    int membern;
public:
    friend void fun(first, second);    // friend prototype in class first
};

class second
{
private:
    int mem1;
    int mem2;
public:
    friend void fun(first, second);    // friend prototype in class second
};

void fun(first o1, second o2)          // definition: NO "friend" keyword
{
    cout << o1.member1 << o1.membern;  // access class first private data
    cout << o2.mem1    << o2.mem2;     // access class second private data
}

int main()
{
    first  obj1;
    second obj2;
    fun(obj1, obj2);    // simple call, cannot use dot operator
    return 0;
}
```

5.4 Friend Classes

A **friend class** allows all member functions of one class to access the private and public members of another class directly.

To make second a friend of first, write inside first:

```
class first
{
private:
    friend class second;   // second can access first's private members
    int member1;
    int membern;
public:
    first() { member1 = 10; membern = 20; }
};

class second
{
public:
    void fun(first o1)
    {
        // second is friend of first, so can access member1 and membern
        cout << o1.member1 << " " << o1.membern << endl;
    }
};

int main()
{
    first  obj1;
    second obj2;
    obj2.fun(obj1);    // obj2 calls function, passing obj1 as argument
    // obj1.fun() would FAIL — first does not have fun()
    return 0;
}
```

■■ COMMON MISTAKES

- Friendship is NOT mutual by default. If second is a friend of first, first is NOT automatically a friend of second.
- Friendship is NOT inherited. A derived class does not automatically get the friend privileges of its parent.
- Do NOT write 'friend' in the function definition — only in the prototype inside the class.

Friend Function vs Member Function	Friend Class vs Regular Class
Not a member of the class.	All its functions can access another class's private data.
Called like a regular function (no dot operator).	Must be declared as friend inside the class being accessed.
Declared with 'friend' keyword in the class.	Friendship declared with 'friend class ClassName'.
Can access private members of the class.	One-sided: friendly class can access, not vice versa.

■ **EXAM-FOCUSED POINTS**

- **What is a friend function? How does it differ from a member function?**
- **Can access specifiers restrict a friend function? Explain.**
- **Where is the 'friend' keyword written — in prototype or definition?**
- **What is a forward declaration and when is it needed?**
- **Why do some programmers argue friend functions violate OOP principles?**
- **Is friendship in C++ mutual and inherited? Justify your answer.**

QUICK REVISION SUMMARY

Concept	Summary
`Class`	Blueprint / template for creating objects. Contains data members and member functions.
`Object`	An instance of a class. Has its own copy of all data members.
`Data Member`	Variables inside a class. Private by default. Represent the state of an object.
`Member Function`	Functions inside a class. Define the behaviour of an object.
`Encapsulation`	Bundling data + functions in one unit (class) while hiding internal data.
`Scope Resolution (::)`	Links a function definition (outside class) back to the class it belongs to.
`const Function`	Read-only member function. Cannot modify any data member.
`public`	Accessible anywhere inside and outside the class.
`private`	Accessible only inside the class. Default for class members.
`Constructor`	Auto-called on object creation. Same name as class. No return type.
`Destructor`	Auto-called when object goes out of scope. ~ClassName(). No args, no return.
`Nullary Constructor`	Constructor with no parameters.
`Parameterized Constructor`	Constructor that accepts arguments.
`Copy Constructor`	Creates new object from existing one. obj2(obj1) or obj2 = obj1.
`Shallow Copy`	Member-by-member copy. Default behaviour. Unsafe for pointer members.
`Deep Copy`	Allocates new memory for pointer members. Each object is fully independent.
`Friend Function`	Regular function with access to private members. Not a class member.
`Friend Class`	A class whose functions can access another class's private members.
`Forward Declaration`	Tells compiler a class exists before its full definition.

End of OOP Complete Exam Notes — Good Luck!